

MPE Scope & Change Control

Status: MANDATORY

Version: 2026-09-24

Canonical source:

`Murkin1980/murat-project-engineer/docs/governance/SCOPE-CHANGE-CONTROL.md`

This policy defines the default execution discipline for Murat Project Engineer repositories and coding agents.

Project-specific rules MAY be stricter. They MUST NOT silently weaken this policy.

1. Primary principle

Perform the task with the smallest sufficient correct change.

The goal is the requested behavior with the minimum necessary new code, dependencies, infrastructure, configuration, abstractions, and unrelated edits.

Simplicity first.

Explicit current owner instructions take priority unless they conflict with fundamental project rules, security requirements, an approved architecture invariant, or the deep-change gate.

2. Source of truth

For the current task, use this priority order:

1. Explicit current owner instruction.
2. Current approved checkpoint/spec.
3. Repository-local mandatory governance and project instructions.
4. Existing architecture/product/domain contracts.
5. Existing code and tests.
6. Historical conventions and prior decisions.

If sources materially conflict, do not silently choose the convenient interpretation. Surface the conflict and follow the higher-priority source unless a deep-change decision is required.

3. Checkpoint is the task boundary

When work is governed by a checkpoint/spec:

- implement only that checkpoint;
- do not invent the next checkpoint;
- do not prebuild later work;
- do not turn a discovered opportunity into current scope;
- stop when the approved checkpoint is complete;
- if the spec says `STOP`, stop.

A new checkpoint begins only after an owner instruction or committed owner-approved spec authorizes it.

4. Before editing

Inspect only the code and call paths needed to make the change safely.

Do not explore the whole repository for a small task.

Determine:

- where the behavior lives;
- which call path is relevant;
- which existing component/pattern already solves similar work;
- which tests cover the behavior;
- which architecture constraints apply.

Make a small obvious fix directly. Use a short plan only when the approach is unclear, multiple subsystems are affected, or consequences are substantial.

Resolve ordinary implementation details independently.

5. New Idea Filter

Before implementing any new substantial idea, product, feature, service, agent, plugin, integration, automation, or repository, evaluate:

- can an active project be extended;
- can an existing component be reused;
- does this duplicate existing capability or infrastructure;
- is there measurable business value;
- can it be tested as a minimal experiment/MVP;
- what is its priority relative to active projects;
- is it a deep-change.

Allowed primary decisions:

`EXTEND\_EXISTING`

`REUSE\_COMPONENT`

`MERGE`

`EXPERIMENT`

`HOLD`

`NEW\_REPOSITORY`

`REJECT`

Default preference: strengthen existing systems before creating parallel ones.

The filter is required for substantial new ideas, not for every routine bug fix.

6. Deep-change gate

Do not execute a substantial change without explicit approval when it:

- changes fundamental architecture;
- breaks an approved invariant;
- changes a public contract;
- requires data migration;
- changes the source of truth;
- introduces a new infrastructure platform;
- creates a new repository;
- materially changes security boundaries;
- creates unapproved recurring cost;
- is difficult to reverse;
- exceeds the approved checkpoint boundary.

Stop only the deep-change portion. Continue safe work that remains inside the already approved scope.

7. Implementation order

Before adding new code, check in this order:

1. Existing code and established project patterns.
2. Standard library / built-in platform capability.
3. Already-installed dependencies.
4. Existing internal reusable components.

5. Only then add new code or a new dependency.

Do not create a new abstraction when the existing mechanism solves the task sufficiently.

8. Minimal diff

For a small task, prefer a targeted edit.

Avoid unless required:

- unrelated refactors;
- large renames;
- file moves;
- rewriting working modules;
- extra architecture layers;
- new configuration systems;
- generic frameworks for one use case;
- speculative future-proofing.

Every changed line should have a clear relationship to the current task.

9. Fix the root cause

Do not hide a problem behind extra checks, fallback layers, or workarounds when the source can be corrected safely.

Root-cause work is not permission for a broad refactor.

Choose the smallest change that genuinely fixes the cause.

10. Do not design hypothetical future needs

Do not add:

- extensibility without a real consumer;
- a generic API without a second concrete use case;
- unused settings;
- compatibility layers without a current requirement;
- adapter/interface/factory layers only for possible future work;
- infrastructure for an unapproved feature.

Future opportunity is separate scope.

11. Dependencies

Add a dependency only when current capabilities are insufficient and the dependency meaningfully reduces complexity or risk for the current task.

Before adding one, answer:

Why can this not be solved reasonably with the existing stack?

If there is no convincing answer, do not add it.

12. Adjacent problems

Fix a neighboring issue now only when it:

- blocks the current task;
- prevents correct verification;
- was directly caused by the current change.

Otherwise record it briefly for separate work and keep current scope closed.

13. Replaced implementations

Remove superseded code, dead branches, temporary files, commented-out implementations, and debug artifacts created by the task.

Before deleting an older implementation, verify that it is not required for:

- public contract compatibility;
- backward compatibility;
- migration;
- an approved fallback;
- a supported interface/version.

14. Preserve system protections

Minimal change must not weaken necessary:

- input validation;
- error handling;
- security;
- project/tenant isolation;
- idempotency contracts;

- accessibility;
- required observability;
- runtime invariants.

15. When to ask

Continue implementation, verification, and correction inside already-approved authority.

Do not repeatedly ask whether to continue, run required tests, or fix an obvious implementation error.

Ask or stop before:

- substantial scope expansion;
- architecture/deep-change;
- a new repository;
- unapproved spending;
- irreversible action;
- additional permissions;
- materially different product interpretations.

If the task is analysis/review only, do not modify code.

16. Testing

After a change, verify the requested behavior and its consequences.

Order:

1. Existing relevant tests.
2. Mandatory project checks.
3. New tests only for real observable behavior or regression risk.

Test contracts and behavior, not implementation trivia.

Do not repeatedly rerun unchanged successful checks without a concrete reason.

Repeat when new edits, failures, merge/rebase effects, or a specific unresolved doubt justify it.

17. Do not code to the test

Tests verify the contract; they are not the product.

If a test conflicts with newly approved behavior, verify the spec first, then update the implementation and test to match the approved contract.

Do not add test-only production behavior.

18. If the plan grows

If work starts producing new layers, infrastructure, frameworks, broad refactors, extra features, generalized systems, many new files, or repeated checks, compare them with the original requirement.

Ask:

Is this necessary to complete the current task?

If not, remove it from the change.

19. No parallel systems by default

Before creating a new service, worker, API, storage layer, dashboard, adapter, workflow, agent, utility, library, or repository, check whether an existing system can be extended or reused.

A parallel system requires explicit justification and the New Idea Filter.

20. Evidence

After substantial work, leave enough evidence in the project's established format for the next executor to continue without rediscovery.

As applicable, include:

- what changed;
- files/components affected;
- test/typecheck/build results;
- commit SHA / branch / PR;
- deploy status;
- known limitations;
- next authorized action.

Do not create duplicate evidence formats when the repository already defines one.

21. Change-size and merge-pressure control

Line count is a risk signal, not a productivity target and not a reason to interrupt correct work mid-change.

Measure ****meaningful changed lines**** as additions + deletions on the current branch/PR relative to its merge base with the default branch.

Exclude from the threshold when they are predominantly mechanical and separately reviewable:

- lock files;
- generated code or generated artifacts;
- vendored third-party files;
- snapshots;
- large fixtures, captures, datasets, evidence payloads, or compiled/minified output;
- formatting-only bulk changes.

Do not split coherent code merely to stay below a number. Do not stop in the middle of an atomic edit, migration, schema transition, repair, or verification cycle.

Use these thresholds as merge pressure:

- **0-1,500 meaningful changed lines · NORMAL.**** Continue inside the approved scope.
- **1,500-3,000 · WATCH.**** Keep scope tight. Prefer finishing the current atomic task before taking adjacent work.
- **>=3,000 · MERGE CHECKPOINT.**** Do not interrupt the current atomic task. At the next stable, testable boundary, merge the current atomic task.
- **>=5,000 · STRONG MERGE PRESSURE.**** Finish the already-started bounded task and verification, but do not merge.
- **>=10,000 meaningful handwritten/semantic lines · EXCEPTION.**** Continue only when the change is genuinely necessary.

These thresholds are ****not hard stop signals for an external coder who is in the middle of valid work****. The safe sequence is:

1. complete the current atomic operation to a coherent state;
2. make the state testable and resumable;
3. run the required verification;
4. record evidence/handoff;
5. then integrate, split, or request the required decision before expanding scope.

A checkpoint may be merged while its overall outcome is ``BLOCKED`` when the completed implementation is coherent, tests/checks pass, unresolved blockers are external or explicitly isolated, and merging does not weaken the default branch.

Changed-file count is a secondary warning signal: around ****25+ meaningful files**** should prompt the same scope/diff review, but it is not an automatic

stop when the files are mechanically related.

Never optimize for a smaller line count by hiding complexity, compressing code, reducing tests, weakening evidence, or combining unrelated concerns.

22. Merge and deploy authority

Successful implementation does not automatically grant merge/deploy authority.

Merge or deploy automatically only when that authority is already granted by the owner or project workflow.

If authority exists, do not ask again after every technical check.

If not, stop at the permitted boundary and preserve evidence.

23. Definition of done

The task is done when:

- requested behavior works;
- the root cause is addressed;
- scope remains within the approved boundary;
- mandatory verification passes;
- necessary tests are updated;
- temporary artifacts are removed;
- superseded unnecessary code is removed safely;
- no known blocker remains inside current scope;
- required evidence is recorded.

Do not stop at a plan, partial implementation, or the first green test.

24. Final report

Report briefly:

****RESULT:**** PASS / BLOCKED / PARTIAL

****Changed:**** what actually changed.

****Verified:**** checks and results.

****Remaining:**** only real unresolved items.

If complete, do not invent the next checkpoint.

Short form

****Read scope rules first -> respect the checkpoint -> reuse before building -> make the smallest correct change -> fix the root cause -> verify real behavior -> leave evidence -> stop at the approved boundary.****